Dynamic Partition Pruning (DPP)

Prepared By: Kaviraj T U

→ Spark Optimization That Can 10× Your Query Performance

👉 In many data lake pipelines, tables are **partitioned** by columns like:

- ✓ date
- ✓ region
- ✓ country

👉 Partitioning helps Spark **avoid scanning unnecessary data**.

👉 But sometimes Spark still reads **many partitions that are not needed**.

👉 This is where **Dynamic Partition Pruning (DPP)** becomes extremely powerful.

👉 It allows Spark to **skip irrelevant partitions at runtime**, drastically reducing the amount of data scanned.

The Problem Without Dynamic Partition Pruning

👉 Consider two tables:

✓ **Fact Table**

sales (partitioned by date)

✓ **Dimension Table**

promotions

👉 **Example query:**

```
SELECT *
FROM sales s
JOIN promotions p
ON s.date = p.date
WHERE p.region = 'US'
```

👉 Without optimization, Spark may:

- Scan **all partitions of the sales table**
- Then perform the join
- Finally filter based on the condition

👉 This leads to **unnecessary data scanning**.

⚡ What Dynamic Partition Pruning Does

👉 Dynamic Partition Pruning allows Spark to:

Read the **filtered dimension table first**
Identify relevant partition values
Prune unnecessary partitions from the large fact table

👉 **Execution becomes:**

Filter promotions ➜ find relevant dates
↓
Scan only matching sales partitions
↓
Perform join

👉 This can reduce data scanning **dramatically**.

Example Scenario

👉 **Fact table:**

➜ **sales**

➜ Partitions = 365 (one per day)

👉 Query filters only:

➜ date = '2024-11-25'

👉 **Without DPP:**

365 partitions scanned

👉 **With DPP:**

1 partition scanned

👉 Huge performance improvement.

How Spark Executes Dynamic Partition Pruning

👉 When Spark detects a join on a partition column:

- It builds a **subquery** from the filtered dimension table
- Uses the results to determine which partitions to read
- Applies pruning before scanning the large dataset

👉 This optimization is applied **during query execution**, not during initial planning.

Enabling Dynamic Partition Pruning

👉 In Spark 3+, it is usually enabled by default.

👉 Key configuration:

📤 **spark.conf.set("spark.sql.optimizer.dynamicPartitionPruning.enabled", "true")**

👉 To verify usage, check the **Spark physical plan**:

df.explain(True)

👉 Look for:

DynamicPruningSubquery

⚠️ When DPP Works Best

👉 Dynamic Partition Pruning works well when:

- Fact tables are **partitioned**
- Join condition uses the **partition column**
- Dimension tables are **filtered**
- Fact tables are **very large**

👉 Common example:

Fact table → billions of rows
Dimension table → thousands of rows

Real Production Insight

👉 Many Spark jobs scan **far more data than necessary**.

👉 Dynamic Partition Pruning is one of the easiest ways to:

✔ Reduce I/O
✔ Reduce shuffle size
✔ Improve query latency
✔ Lower compute cost

👉 In some workloads, DPP can improve performance by **5×–10×**.

Final Takeaway

👉 Partitioning alone is not enough to optimize large data lake queries.

👉 Dynamic Partition Pruning makes partitioning **intelligent at runtime**, allowing Spark to read only the data that actually matters.

👉 For large fact tables, this feature can be a **game-changer for query performance**.

Thank You.!

Happy Learning...